

General

通用

本页面介绍 Testlib checker/interactor/validator 的一些通用状态/对象/函数、一些用法及注意事项。请在阅读其他页面
前完整阅读本页面的内容。

通用状态

结果	Testlib 别名	含义
Ok	<code>_ok</code>	答案正确。
Wrong Answer	<code>_wa</code>	答案错误。
Presentation Error	<code>_pe</code>	答案格式错误。注意包括 Codeforces 在内的许多 OJ 并不区分 PE 和 WA。
Partially Correct	<code>_pc(score)</code>	答案部分正确。仅限于有部分的测试点，其中 <code>score</code> 为一个正整数，从（没分）到（可能的最大分数）。（ <code>quitf+_pc</code> 只是为了兼容旧的 <code>pascal-testlib</code> ，如果想要输出部分分，建议使用 <code>quitp¹</code> ）
Fail	<code>_fail</code>	<code>validator</code> 中表示输入不合法，不通过校验。 <code>checker</code> 中表示程序内部错误、标准输出有误或选手输出比标准输出更优，需要裁判/出题人关注。（也就是题目锅了）

通常程序的返回值表明结果，但是也有一些其他方法：创建一个输出 xml 文件、输出信息到 `stdout` 或其他位置.....
这些都通过下方函数表中的 `quitf` 函数来完成。

通用对象

对象	含义
<code>inf</code>	输入文件流
<code>ouf</code>	选手输出流
<code>ans</code>	参考输出流

通用函数

非成员函数：

调用	含义
<code>void registerTestlibCmd(int argc, char* argv[])</code>	注册程序为 <code>checker</code>
<code>void registerInteraction(int argc, char* argv[])</code>	注册程序为 <code>interactor</code>
<code>void registerValidation()/void registerValidation(int argc, char* argv[])</code>	注册程序为 <code>validator</code>

调用	含义
void registerGen(int argc, char* argv[], int randomGeneratorVersion)	注册程序为 generator randomGeneratorVersion 推荐为 1
void quit(TResult verdict, string message)/void quitf(TResult verdict, string message, ...)	结束程序, 返回 verdict, 输出 message
void quitif(bool condition, TResult verdict, string message, ...)	如果 condition 成立, 调用 quitf(verdict, message, ...)
void quitp(F points, string message, ...)	结束程序, 返回部分分。大部分 OJ (如洛谷、UOJ) 的 points 需要提供一个内的实数, 表示得分百分比, 还有部分 OJ (如 Lyrio) 的 points 需要提供一个的实数 (OJ 会自动舍弃小数部分), 表示百分制下的测试点得分

流成员函数:

调用	含义
char readChar()	读入一个字符
char readChar(char c)	读入一个字符, 必须为 c
char readSpace()	等同于 readChar(' ')
string readToken()/string readWord()	读入一个串, 到空白字符 (空格、Tab、EOLN 等) 停止
string readToken(string regex)/string readWord(string regex)	读入一个串, 必须与 regex 匹配
long long readLong()	读入一个 64 位整数
long long readLong(long long L, long long R)	读入一个 64 位整数, 必须在 之间
vector<long long> readLongs(int n, long long L, long long R)	读入 n 个 64 位整数, 必须均在 之间
int readInt()/int readInteger()	读入一个 32 位整数
int readInt(int L, int R)/int readInteger(L, R)	读入一个 32 位整数, 必须在 之间
vector<int> readInts(int n, int L, int R)/vector<int> readIntegers(int n, int L, int R)	读入 n 个 32 位整数, 必须均在 之间
double readReal()/double readDouble()	读入一个双精度浮点数
double readReal(double L, double R)/double readDouble(double L, double R)	读入一个双精度浮点数, 必须在 之间
double readStrictReal(double L, double R, int minPrecision, int maxPrecision)/double readStrictDouble(double L, double R, int minPrecision, int maxPrecision)	读入一个双精度浮点数, 必须在 之间, 小数位数必须在 之间, 不得使用指数计数法等非正常格式
string readString()/string readLine()	读入一行 (包括换行符), 同时将流指针指向下一行的开头
string readString(string regex)/string readLine(string regex)	读入一行, 必须与 regex 匹配
void readEoln()	读入 EOLN (在 Linux 环境下读入 LF, 在 Windows 环境下读入 CR LF)
void readEof()	读入 EOF
void quit(TResult verdict, string message)/void quitf(TResult verdict, string message, ...)	结束程序, 若 Stream 为 ouf 返回 verdict, 否则返回 _fail; 输出 message
void quitif(bool condition, TResult verdict, string message, ...)	如果 condition 成立, 调用 quitf(verdict, message, ...)

未完待续.....

极简正则表达式

上面的输入函数中的一部分允许使用「极简正则表达式」特性，如下所示：

- 字符集。如 `[a-z]` 表示所有小写英文字母，`[^a-z]` 表示除小写英文字母外任何字符。
- 范围。如 `[a-z]{1,5}` 表示一个长度在 范围内且只包含小写英文字母的串。
- 「或」标识符。如 `mike|john` 表示 `mike` 或 `john` 其一。
- 「可选」标识符。如 `-?[1-9][0-9]{0,3}` 表示 范围内的非零整数（注意那个可选的负号）。
- 「重复」标识符。如 `[0-9]*` 表示零个或更多数字，`[0-9]+` 表示一个或更多数字。
- 注意这里的正则表达式是「贪婪」的（「重复」会尽可能匹配）。如 `[0-9]?1` 将不会匹配 `1`（因为 `[0-9]?` 将 `1` 匹配上，导致模板串剩余的那个 `1` 无法匹配）。

首先 include testlib.h

请确保 `testlib.h` 是你 `include` 的 第一个 头文件，`Testlib` 会重写/禁用（通过名字冲突的方式）一些与随机有关的函数（如 `random()`），保证随机结果与环境无关，这对于 `generator` 非常重要，[generator 页面](#) 会详细说明这一点。

使用项别名

推荐给 `readInt/readInteger/readLong/readDouble/readWord/readToken/readString/readLine` 等的有限制调用最后多传入一个 `string` 参数，即当前读入的项的别名，使报错易读。例如使用 `inf.readInt(1, 100, "n")` 而非 `inf.readInt(1, 100)`，报错信息将为 `FAIL Integer parameter [name=n] equals to 0, violates the range [1, 100]`。

使用 ensuref/ensure()

这两个函数用于检查条件是否成立（类似于 `assert()`）。例如检查，我们可以使用

```
1 ensuref(x[i] != y[i], "Graph can't contain loops");
```

还可以使用 C 风格占位符如

```
1 ensuref(s.length() % 2 == 0,
2         "String 's' should have even length, but s.length()=%d",
3         int(s.length()));
```

它有一个简化版 `ensure()`，我们可以直接使用 `ensure(x > y)` 而不添加说明内容（也不支持添加说明内容），如果条件不满足报错将为 `FAIL Condition failed: "x > y"`。很多情况下不加额外的说明的这种报错很不友好，所以我们通常使用 `ensuref()` 并加以说明内容，而非使用 `ensure()`。

▼ Warning

注意全局与成员 `ensuref/ensure()` 的区别

全局函数 `::ensuref/ensure()` 多用于 `generator` 和 `validator` 中，如果检查失败将统一返回 `_fail`。

成员函数 `InStream::ensuref/ensure()` 一般用于判断选手和参考程序的输出是否合法。当 `InStream` 为 `ouf` 时，返回 `_wa`；为 `inf`（一般不在 `checker` 中检查输入数据，这应当在 `validator` 中完成）或 `ans` 时，返回 `_fail`。详见 [Checker-编写 readAns 函数](#)。

本文主要翻译并综合自 [Testlib - Codeforces](#) 系列。`testlib.h` 的 GitHub 存储库为 [MikeMirzayanov/testlib](#)。

本页面最近更新：2024/9/6 20:56:11, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [ayuusweetfish](#), [CCXXI](#), [Chrogeek](#), [countercurrent-time](#), [cubercsl](#), [Enter-tainer](#), [HeRaNO](#), [NachtgeistW](#), [ouuan](#), [ouuan](#), [shuzhouliu](#), [StudyingFather](#), [SukkaW](#), [Xeonacid](#), [ZnPdCo](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用